Hybrid Deep Learning and Traditional Techniques

Goal: Build a stronger, more efficient disease detection model by **hybridizing** (mixing) different techniques.

Examples:

- CNN + Vision Transformer (e.g., Jute leaf, Tomato pathology hybrid)
- Lightweight CNN + optimization/ensemble (e.g., agricultural leaf diseases)
- CNN + XGBoost/FFNN (e.g., endoscopy for GERD/Polyp)
- Hybrid for H&E pathology (e.g., Cervical AIS identification)

Phase 0 – Preparation (Before Starting)

□ Decide what kind of hybrid you want to try today

- CNN backbone + Transformer (local + global features)
- CNN + ML classifier (e.g., CNN features → XGBoost/SVM)
- Lightweight hybrid (e.g., MobileNet + attention/optimized blocks)
- Others

□ Choose dataset folder structure Examples:

```
├── train
│   ├── Healthy
│   ├── Diseased_Class1 (e.g., Jute_Cercospora)
│   └── …
├── val
└── test
```

or (for medical like H&E / endoscopy)

```
├── Normal
├── GERD
├── Polyp
├── AIS etc.
```

□ Decide number of classes: _____ (e.g., 2, 4, multi-class)

□ Choose backbone/components for your hybrid Common choices:

- CNN part: ResNet50, EfficientNet, MobileNetV3, DenseNet
- Other parts: ViT (small), XGBoost, SVM, Attention blocks

Phase 1 – Data Preparation Checklist

□ Images resized to standard size (224×224 or 256×256)

□ Normalized (ImageNet stats or dataset-specific)

□ Augmentation ON for training (rotate, flip, brightness, etc.)

□ Class imbalance checked → decided:

- Weighted loss / sampler
- Focal loss (good for medical/agri rare classes)
- Ignore for first run □ Train / Val / Test split ready (80/10/10 or similar) □ Number of images noted: Train: _____ Val: _____ Test: _____

Phase 2 – Choose & Prepare ONE Experiment

Experiment A – CNN + Transformer hybrid (e.g., ResNet + ViT layers)
Experiment B – CNN feature extractor + ML classifier (e.g., EfficientNet → XGBoost)
Experiment C – Lightweight hybrid (e.g., MobileNet + attention/optimized blocks)
Experiment D – Ensemble hybrid (multiple CNNs fused + final classifier) or others

Phase 3 – Model Building Steps
□ Step 3.1 – Load pre-trained CNN backbone
□ Step 3.2 – Freeze early layers (transfer learning)
□ Step 3.3 – Add hybrid part:
- Transformer/attention layers OR
- ML head (extract features → feed to XGBoost/SVM) OR
- Lightweight blocks (depthwise conv, SE, etc.)

□ Step 3.4 – Change final layer to match your num_classes
□ Step 3.5 – Add any special: dropout, focal loss, class weights
□ Step 3.6 – Decide loss: CrossEntropy / Focal / Weighted
□ Step 3.7 – Optimizer: Adam / AdamW / SGD
□ Step 3.8 – Learning rate: start 1e-4 or 1e-3 (lower for fine-tune)

Phase 4 – Training Workflow (Repeat for Hybrid)
1. Put model on GPU if possible
2. For each epoch (10–50, depend on dataset):
   □ Train full pass on train set □ Get train loss & accuracy
   □ Validate on val set → val loss + F1/accuracy
   □ Save if best val result (early stopping if no improvement)
3. Finish → keep best weights (lowest val loss or best F1)

Phase 5 – Hybrid-Specific Actions
CNN + Transformer
□ Train end-to-end (both parts learn together)
□ Possibly freeze CNN first, then unfreeze
CNN + ML classifier
□ Train CNN as feature extractor (no grad on head)
□ Extract features from val set → train ML model (XGBoost etc.)
□ Tune ML hyperparameters (grid/random search)
Lightweight hybrid
□ Check model size (parameters < 5M ideally)
□ Test inference speed (important for mobile/agri use)

Phase 6 – Final Evaluation (Must Do)
□ Test only once on **test set** with best model
□ Calculate:

Metric	**Value**
Accuracy	____ %
Weighted F1	____
Macro F1	____

Metric	Value

Precision/Recall per class list

□ Compare with:

- Single CNN baseline
- Other hybrids you tried

This is a sample format for Hybrid Model Development. If you have any ideas or suggestions for improving the process, please feel free to share making any changes to the approach. Good Luck